

RAPORT- Projekt nr 6

Cel części podstawowej

Celem tej części projektu było zastosowanie metod optymalizacji ciągłej do rozwiązania problemu regresji logistycznej. W projekcie porównano trzy metody: gradient descent ze stałym krokiem, nieliniową metodę gradientów sprzężonych NCG-PRP+ z backtrackingiem Armijo oraz metodę Newtona-Raphsona z line search.

Dla każdej metody sprawdzono liczbę iteracji, czas działania, końcową wartość funkcji straty oraz dokładność klasyfikacji na zbiorze testowym. Dzięki temu można było porównać, jak różne metody optymalizacji radzą sobie z tym samym problemem uczenia modelu.

Dane i przygotowanie zbioru

W projekcie wykorzystano zbiór `breast_cancer` dostępny w bibliotece `sklearn.datasets`. Jest to zbiór przeznaczony do klasyfikacji binarnej. Dane zawierają 569 obserwacji oraz 30 cech opisujących przypadki medyczne. Model ma za zadanie przypisać każdą obserwację do jednej z dwóch klas.

Na początku dane zostały rozdzielone na macierz cech `X` oraz wektor etykiet `y`. Etykiety początkowo miały wartości `0` i `1`, ale zostały przekształcone do postaci `-1` i `1`. Taki zapis jest wygodniejszy przy implementacji funkcji straty logistycznej.

Następnie dane podzielono na zbiór treningowy i testowy. Zbiór treningowy służył do wyznaczania parametrów modelu, a zbiór testowy do oceny jakości klasyfikacji. Przed uruchomieniem metod optymalizacji wykonano standaryzację cech, ponieważ metody gradientowe są wrażliwe na skalę danych. Po standaryzacji każda cecha ma średnią równą 0 i odchylenie standardowe równe 1. Do macierzy danych dodano także kolumnę jedynek, aby model mógł uwzględnić wyraz wolny.

Funkcja straty logistycznej

Regresja logistyczna została potraktowana jako problem minimalizacji funkcji straty. Funkcja straty określa, jak dobrze aktualne parametry modelu dopasowują się do danych treningowych. Im mniejsza wartość funkcji straty, tym lepiej model dopasowuje się do danych.

W projekcie minimalizowano funkcję:

$$\ell(w) = \frac{1}{m} \sum_{i=1}^m \log(1 + \exp(-y_i x_i^T w))$$
$$\ell(w) = \frac{1}{m} \sum_{i=1}^m \log(1 + \exp(-y_i x_i^T w))$$

W tym wzorze `w` oznacza wektor parametrów modelu, `xi` oznacza wektor cech dla jednej obserwacji, `yi` to etykieta klasy, a `m` to liczba obserwacji treningowych.

W implementacji wykorzystano funkcję `np.logaddexp(0, -z)`, ponieważ pozwala ona stabilnie numerycznie obliczać wartość wyrażenia `log(1 + exp(-z))`. Dzięki temu zmniejsza się ryzyko problemów numerycznych przy dużych wartościach argumentu funkcji wykładniczej.

Gradient i hesjan

Do działania metod optymalizacji potrzebny był gradient funkcji straty. Gradient wskazuje kierunek największego wzrostu funkcji, dlatego przy minimalizacji wykonuje się krok w kierunku przeciwnym do gradientu.

Gradient funkcji straty został zaimplementowany samodzielnie. W kodzie zastosowano również zabezpieczenie numeryczne `np.clip(z, -500, 500)`, aby uniknąć przepełnienia przy obliczaniu funkcji wykładniczej `np.exp(z)`.

Dodatkowo zaimplementowano hesjan, czyli macierz drugich pochodnych funkcji straty. Hesjan jest potrzebny w metodzie Newtona-Raphsona, ponieważ ta metoda korzysta nie tylko z informacji o nachyleniu funkcji, ale także z informacji o jej krzywiznie. Dzięki temu metoda Newtona może wykonywać bardziej dopasowane kroki, chociaż pojedyncza iteracja jest bardziej kosztowna obliczeniowo.

Gradient Descent

Pierwszą zaimplementowaną metodą był gradient descent ze stałym krokiem. Jest to jedna z najprostszych metod optymalizacji. W każdej iteracji obliczany jest gradient funkcji straty, a następnie parametry modelu są aktualizowane w kierunku przeciwnym do gradientu.

W projekcie zastosowano stały krok `alpha = 0.1`. Dla każdej iteracji zapisywano wartość funkcji straty oraz normę gradientu. Algorytm kończył działanie wtedy, gdy norma gradientu była mniejsza od ustalonej tolerancji albo gdy osiągnięto maksymalną liczbę iteracji.

Zaletą gradient descent jest prosta implementacja i niski koszt pojedynczej iteracji. Wadą jest to, że metoda może potrzebować wielu iteracji, szczególnie gdy stały krok nie jest idealnie dobrany.

NCG-PRP+ z backtrackingiem Armijo

Drugą metodą była nieliniowa metoda gradientów sprzężonych w wariancie PRP+. Metoda ta, podobnie jak gradient descent, korzysta z gradientu, ale dodatkowo wykorzystuje informację z poprzednich iteracji. Dzięki temu może tworzyć lepsze kierunki poszukiwań niż zwykły gradient descent.

W pierwszej iteracji kierunek jest przeciwny do gradientu. W kolejnych iteracjach kierunek jest wyznaczany na podstawie aktualnego gradientu oraz poprzedniego kierunku. W wariancie PRP+ współczynnik odpowiedzialny za udział poprzedniego kierunku jest ograniczany od dołu

przez zero. Oznacza to, że jeżeli algorytm wyznaczy niekorzystny kierunek, następuje restart i metoda wraca do kierunku największego spadku.

Długość kroku w metodzie NCG-PRP+ była dobierana za pomocą backtrackingu Armijo. Polega to na tym, że algorytm zaczyna od pewnej długości kroku, a następnie zmniejsza ją, dopóki nie zostanie spełniony warunek wystarczającego spadku funkcji celu. Dzięki temu metoda unika zbyt dużych kroków, które mogłyby zwiększyć wartość funkcji straty.

Metoda Newtona-Raphsona

Trzecią metodą była metoda Newtona-Raphsona. W przeciwieństwie do gradient descent i NCG, metoda Newtona wykorzystuje hesjan, czyli informację o krzywiznie funkcji celu. Dzięki temu może szybciej zbliżać się do minimum.

W każdej iteracji metoda Newtona wyznacza kierunek rozwiązując układ równań liniowych. W implementacji nie obliczano jawnie odwrotności macierzy Hessego, ponieważ jest to mniej stabilne numerycznie. Zamiast tego zastosowano `np.linalg.solve`, co jest poprawniejszym sposobem rozwiązywania takiego układu.

Metoda Newtona zwykle potrzebuje mniej iteracji niż gradient descent, ale pojedyncza iteracja jest droższa, ponieważ wymaga obliczenia hesjanu i rozwiązania układu równań liniowych. W projekcie zastosowano także backtracking line search, aby kontrolować długość kroku.

Wyniki

Wyniki uzyskane dla trzech metod przedstawiono w tabeli.

Metoda	Liczba iteracji	Warunek stopu	Końcowa funkcji	wartość	Accuracy test
Gradient Descent	5000	Nie	0.0445675		0.982456
NCG-PRP+	1000	Nie	0.0410968		0.982456
Newton-Raphson	88	Tak	0.0000737		0.947368

Gradient Descent oraz NCG-PRP+ osiągnęły taką samą dokładność na zbiorze testowym, równą około 0.9825. Obie metody zakończyły działanie po osiągnięciu maksymalnej liczby iteracji, czyli nie spełniły przyjętego warunku stopu opartego na normie gradientu.

Metoda Newtona-Raphsona spełniła warunek stopu po 88 iteracjach i osiągnęła najniższą końcową wartość funkcji straty. Accuracy testowe było jednak niższe niż w przypadku GD i

NCG-PRP+. Może to oznaczać, że metoda Newtona bardzo mocno dopasowała parametry do zbioru treningowego.

Interpretacja wykresów

W notebooku przedstawiono wykres wartości funkcji straty oraz wykres normy gradientu w kolejnych iteracjach. Wykres funkcji straty pokazuje, że wszystkie trzy metody zmniejszają wartość funkcji celu. Gradient Descent zmniejsza funkcję stopniowo, NCG-PRP+ osiąga nieco niższą wartość funkcji niż GD, a metoda Newtona osiąga najmniejszą wartość funkcji straty.

Wykres normy gradientu pokazuje, jak blisko metoda znajduje się punktu stacjonarnego. Im mniejsza norma gradientu, tym bliżej jesteśmy sytuacji, w której dalsza poprawa funkcji celu jest niewielka. W eksperymencie metoda Newtona osiągnęła warunek stopu, natomiast GD i NCG-PRP+ zakończyły działanie przez osiągnięcie maksymalnej liczby iteracji.

Walidacja implementacji

W notebooku wykonano prostą walidację poprawności gradientu. Sprawdzono, czy po wykonaniu małego kroku w kierunku przeciwnym do gradientu wartość funkcji straty maleje. Wynik potwierdził, że gradient został zaimplementowany poprawnie, ponieważ po takim kroku wartość funkcji celu była mniejsza niż przed krokiem.

Taka walidacja jest prostym przykładem sprawdzenia, czy implementacja gradientu zachowuje się zgodnie z intuicją matematyczną.

Wnioski

W części podstawowej projektu udało się zaimplementować pełny pipeline regresji logistycznej: przygotowanie danych, funkcję straty, gradient, hesjan oraz trzy metody optymalizacji. Wszystkie metody uzyskały wysoką dokładność klasyfikacji na zbiorze testowym.

Gradient Descent okazał się najprostszą metodą, ale wymagał największej liczby iteracji. NCG-PRP+ również nie spełnił warunku stopu w zadanym limicie iteracji, ale uzyskał nieco niższą wartość funkcji straty niż GD i taką samą dokładność testową. Metoda Newtona-Raphsona potrzebowała najmniej iteracji i osiągnęła najniższą wartość funkcji straty, ale jej accuracy testowe było niższe niż dla pozostałych dwóch metod.

Na podstawie wyników można stwierdzić, że metody pierwszego rzędu, takie jak GD i NCG, mogą dawać bardzo dobre wyniki klasyfikacji bez konieczności liczenia hesjanu. Metoda Newtona jest natomiast bardzo skuteczna w szybkim zmniejszaniu funkcji straty, ale jest bardziej kosztowna obliczeniowo i może silnie dopasowywać się do danych treningowych.

Część rozszerzona

Poniższa część jest dopisana do wcześniejszej części podstawowej i korzysta z tego samego przygotowania danych, tej samej funkcji straty logistycznej, gradientu, hesjanu oraz funkcji pomocniczych. Wcześniejsza część raportu nie została zmieniona. Rozszerzenie uzupełnia projekt o elementy wymagane na ocenę bardzo dobrą: porównanie pamięci NCG i Newtona, analizę wariantów NCG, regularyzację L2 oraz porównanie z biblioteką referencyjną sklearn.

Memory footprint NCG i Newtona

NCG jest metodą pierwszego rzędu. W praktyce przechowuje głównie kilka wektorów: wektor wag, gradient oraz kierunek poszukiwań. Z tego powodu pamięć potrzebna na część optymalizacyjną rośnie liniowo, czyli $O(n)$.

Metoda Newtona-Raphsona korzysta z hesjanu, czyli macierzy drugich pochodnych o rozmiarze $n \times n$. Jej koszt pamięciowy rośnie więc kwadratowo, czyli $O(n^2)$. Dla zbioru `breast_cancer` liczba parametrów jest mała, ale przy dużych modelach ta różnica staje się bardzo istotna.

Tabela 1. Porównanie przybliżonego zużycia pamięci.

Liczba parametrów	NCG $O(n)$ [MB]	Newton $O(n^2)$ [MB]	Newton / NCG
31	0.000710	0.007805	11
1000	0.022888	7.6447	334
10000	0.228882	763.09	3334

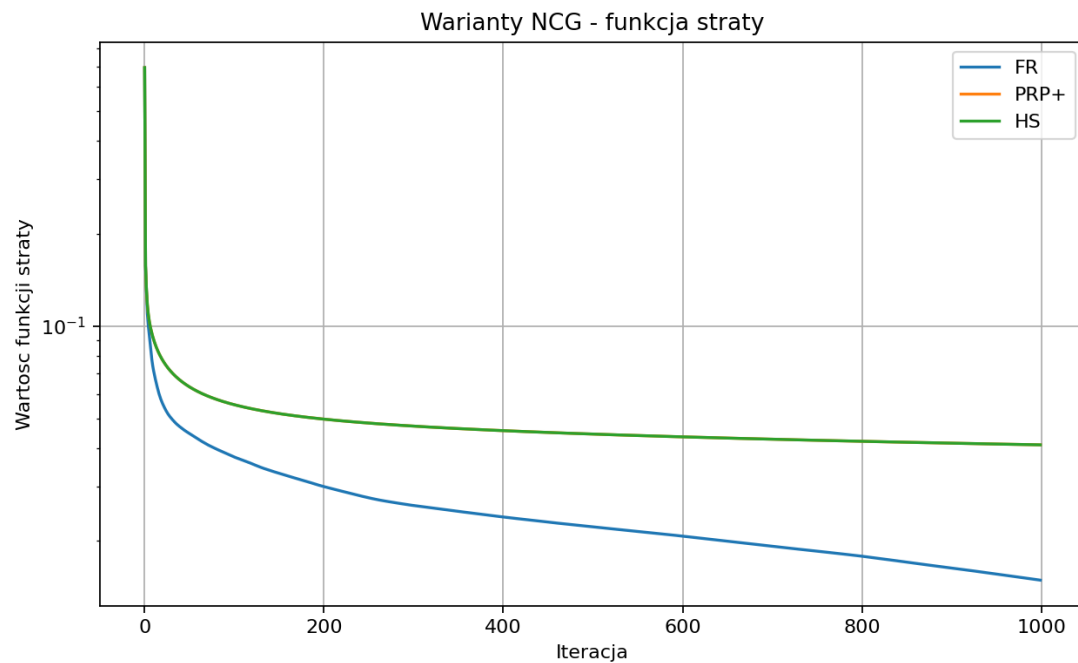
Dla 31 parametrów, czyli dla naszego przykładu po dodaniu wyrazu wolnego, obie metody są pamięciowo tanie. Różnica dobrze widoczna jest jednak w przykładach większych wymiarów. Dla 10000 parametrów NCG potrzebuje tylko kilku wektorów, natomiast Newton musiałby przechowywać hesjan zajmujący setki megabajtów. To wyjaśnia, dlaczego w praktycznych problemach uczenia maszynowego NCG może być akceptowalną alternatywą dla metod drugiego rzędu.

Warianty NCG: FR, PRP+ i HS

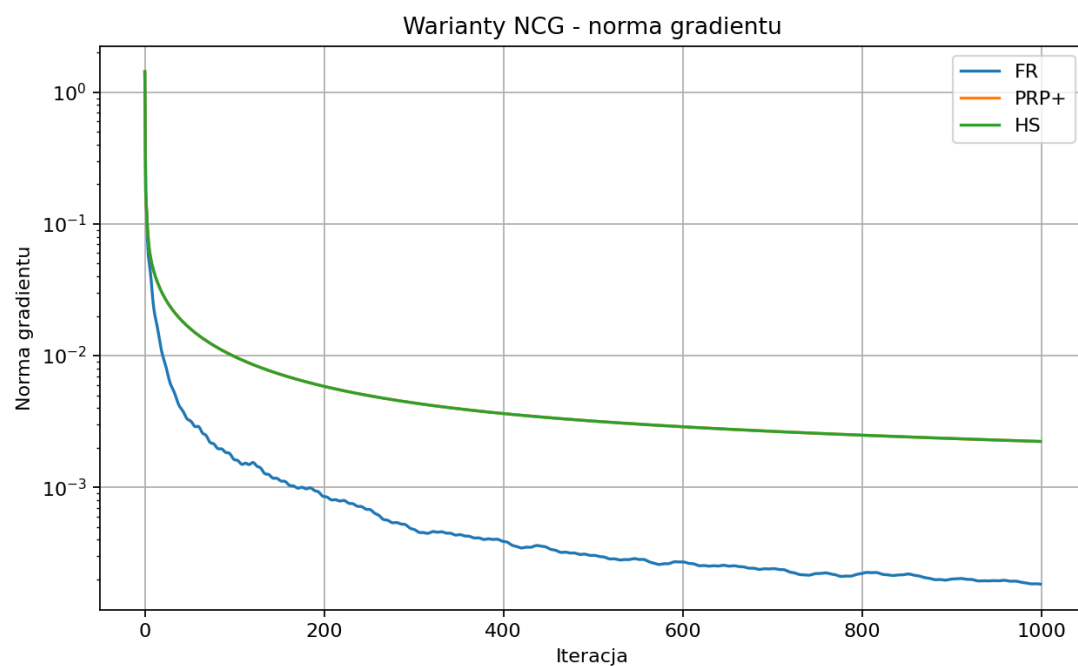
W części podstawowej został użyty wariant PRP+. W rozszerzeniu porównano trzy warianty nieliniowej metody gradientów sprzężonych: Fletcher-Reeves, Polak-Ribiere+ oraz Hestenes-Stiefel. Wszystkie korzystają z tego samego backtrackingu Armijo, aby porównanie dotyczyło głównie sposobu wyznaczania współczynnika beta i kierunku poszukiwań.

Tabela 2. Porównanie wariantów NCG.

Wariant	Iteracje	Stop	Czas CPU [s]	Strata końcowa	Accuracy test	Liczba restartów
FR	1000	Nie	0.139530	0.014906	0.956140	0
PRP+	1000	Nie	0.140797	0.041097	0.982456	1000
HS	1000	Nie	0.144751	0.041097	0.982456	1000



Rysunek 1. Wartość funkcji straty dla wariantów NCG.



Rysunek 2. Norma gradientu dla wariantów NCG.

Warianty PRP+ i HS uzyskały w tym eksperymencie taką samą dokładność testową jak metoda NCG z części podstawowej. Wariant FR osiągnął niższą wartość funkcji straty, ale jego accuracy testowe było słabsze. Pokazuje to, że minimalizacja funkcji celu na zbiorze treningowym nie zawsze przekłada się bezpośrednio na najlepszą jakość klasyfikacji na zbiorze testowym.

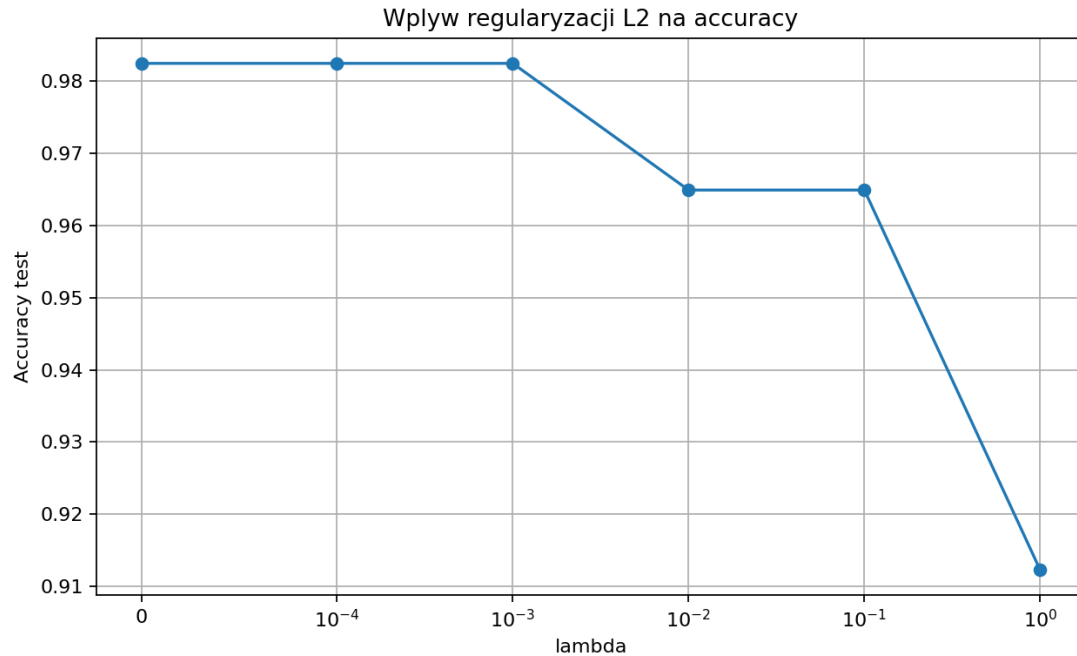
PRP+ jest wariantem praktycznie wygodnym, ponieważ obcięcie beta od dołu przez zero działa jak automatyczny restart. Gdy algorytm wyznaczy niekorzystny kierunek, metoda wraca do kierunku największego spadku. Dzięki temu PRP+ jest zwykle bardziej stabilny niż warianty bez takiego zabezpieczenia.

Regularyzacja L2

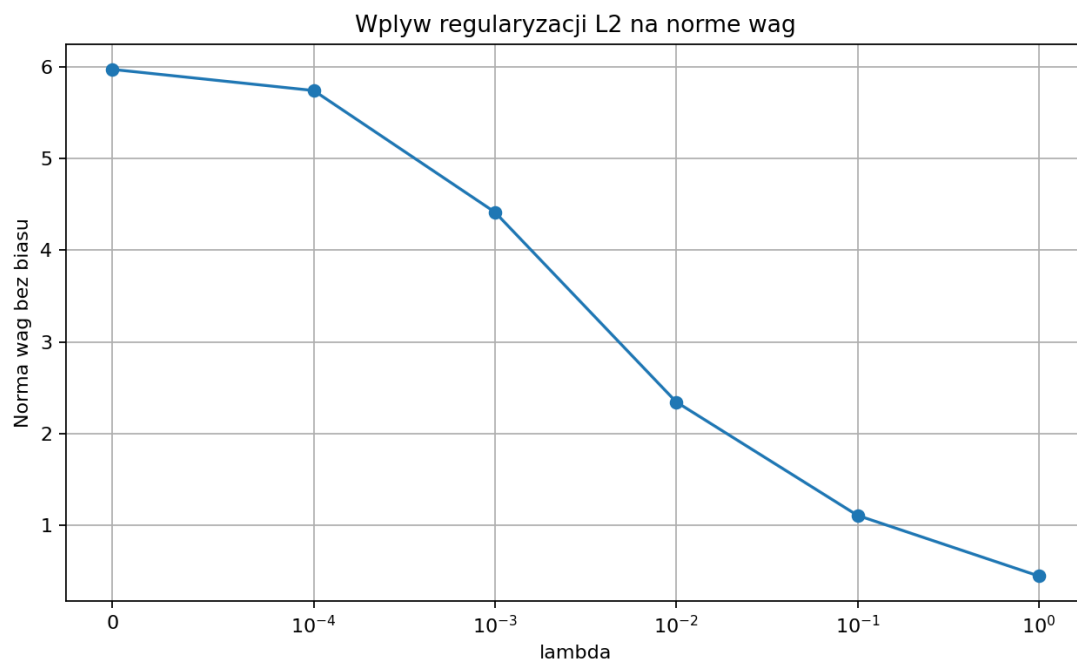
Następnie dodano regularyzację L2 do funkcji straty. Funkcja celu ma wtedy postać $l_{\lambda}(w) = l(w) + \lambda / 2 * ||w||^2$. W implementacji nie karano wyrazu wolnego, czyli pierwszej współrzędnej wektora wag. Celem regularyzacji jest ograniczenie zbyt dużych wartości wag i zmniejszenie ryzyka przeuczenia.

Tabela 3. Wpływ regularyzacji L2 na wynik NCG-PRP+.

Lambda	Iteracje	Stop	Strata bez kary	Strata z karą	Accuracy test	Norma wag bez biasu
0	1000	Nie	0.041097	0.041097	0.982456	5.9761
0.000100	1000	Nie	0.041592	0.043242	0.982456	5.7449
0.001000	1000	Nie	0.046139	0.055888	0.982456	4.4156
0.010000	745	Tak	0.069100	0.096556	0.964912	2.3434
0.100000	164	Tak	0.134006	0.194993	0.964912	1.1044
1	49	Tak	0.284727	0.382504	0.912281	0.442215



Rysunek 3. Wpływ regularyzacji L2 na accuracy testowe.



Rysunek 4. Wpływ regularyzacji L2 na normę wag.

Wyniki pokazują, że wraz ze wzrostem lambda norma wag maleje. Jest to zgodne z intuicją, ponieważ kara L2 ogranicza duże współczynniki modelu. Dla małych wartości lambda accuracy pozostaje bardzo wysokie. Dla zbyt dużej regularyzacji accuracy spada, ponieważ model staje się zbyt ograniczony i może niedouczać danych.

Porównanie z LogisticRegression ze sklearn

Na koniec własne implementacje porównano z biblioteką referencyjną `sklearn.linear_model.LogisticRegression`. Jest to ważne, ponieważ wymagania ogólne

projektu wskazują, że rozwiązanie powinno zostać porównane z co najmniej jedną biblioteką referencyjną. W sklearn użyto regularyzacji L2 i solvera lbfgs.

Tabela 4. Porównanie własnych metod z LogisticRegression.

Metoda	Accuracy test	Strata treningowa
własne GD	0.982456	0.044568
własne NCG-PRP+	0.982456	0.041097
własny Newton-Raphson	0.947368	0.000074
sklearn LogisticRegression	0.982456	-

Accuracy uzyskane przez sklearn jest porównywalne z wynikami Gradient Descent i NCG-PRP+ z części podstawowej. Różnice w wartości funkcji straty nie są bezpośrednio porównywalne dla sklearn, ponieważ biblioteka stosuje własną implementację, domyślną regularyzację i własne kryteria stopu. Porównanie potwierdza jednak, że przygotowany pipeline regresji logistycznej działa poprawnie.

Wnioski z części rozszerzonej

Część rozszerzona potwierdza, że NCG jest kompromisem między prostym Gradient Descent a metodą Newtona-Raphsona. W porównaniu z GD metoda NCG korzysta z informacji z poprzednich iteracji, dlatego może tworzyć lepsze kierunki poszukiwań. W porównaniu z Newtonem nie wymaga hesjanu, więc jest znacznie tańsza pamięciowo.

Najważniejszą przewagą Newtona jest szybkie zmniejszanie funkcji celu, ale ta metoda wymaga liczenia i przechowywania hesjanu. NCG nie osiąga tak małej wartości funkcji straty jak Newton w tym eksperymencie, ale daje bardzo dobrą accuracy testową i ma mniejszy koszt pamięciowy. Regularyzacja L2 dodatkowo pokazuje, że można kontrolować wielkość wag oraz ograniczać zbyt mocne dopasowanie modelu.